

Final Project

Software Design

Warehouse Management System

Microservices Architecture with CQRS & Blazor WebAssembly

Author: Virag Raul

Group: 30433

1 Problem Definition

This assignment extends the Warehouse Management System from Assignments 1 and 2 by migrating from a monolithic Blazor Server application to a microservices-based architecture. The domain — warehouse inventory, stock tracking, orders, and account management — remains unchanged. The goal is to improve scalability, modularity, and maintainability.

Key architectural changes introduced:

- Decomposition into independent microservices, each with its own database.
- CQRS (Command Query Responsibility Segregation) via MediatR.
- JWT-based authentication shared across all services.
- Blazor WebAssembly replaces Blazor Server as the frontend.
- Template Method pattern for data export; Command pattern for UI actions.
- FluentValidation pipeline for server-side input validation.
- Inter-service HTTP communication for event-driven notifications.
- Live chat.
- Internationalization in Blazor.

2 Analysis Phase Use Cases & Requirements

2.1 Functional Requirements

All requirements from Assignments 1 & 2 & 3 are preserved and extended:

- Users can add, edit, soft-delete, and restore product types (Items).
- When adding an item whose name matches a previously soft-deleted item, the system restores the old record instead of creating a duplicate.
- Users can add stock units in bulk; each unit receives an auto-generated serial number.
- Users can search stock units by product name or serial number.
- Individual units can be marked as Available, Reserved, Shipped, or Returned.
- Data can be exported as JSON, CSV, or XML via the Template Method export pipeline.
- The system sends email notifications after key operations for the logged-in account.
- Authentication uses JWT tokens; the Blazor WASM client stores the token and attaches it to every API request.
- Role-based access: Visitor (read-only), Seller (inventory ops), Admin (full access).
- Internationalization of words on the webpage is important.
- Client to client hub chat.

2.2 Use Cases

UC-1 Login / Register

Actor: Any user

Task: Submit email + password. AccountService validates credentials, returns a signed JWT token.

Result: Token stored in browser localStorage; all subsequent requests carry Authorization: Bearer <token>.

UC-2 Add / Restore Product Type

Actor: Admin, Seller

Task: Submit a new Item name. InventoryService checks for a soft-deleted record with the same name.

Result: If a deleted item exists — it is restored (IsDeleted = false) rather than creating a duplicate. Otherwise, a new Item is created.

UC-3 Add Stock Units

Actor: Admin, Seller

Task: Select a product type and enter quantity; InventoryService generates serial numbers and creates StockUnit rows.

Result: Stock counter incremented; units searchable by serial.

UC-4 Search Stock

Actor: Admin, Seller, Visitor

Task: Enter a product name or partial serial number.

Result: Filtered list of matching Stock Units returned by a MediatR Query handler.

UC-5 Soft Delete

Actor: Admin

Task: Mark individual stock units or entire Item categories as deleted.

Result: Records hidden via EF query filters; restorable via UC-2.

UC-6 Export Data

Actor: Admin, Seller

Task: Select format (CSV / JSON / XML) and click Export.

Result: InventoryService executes the Template Method export pipeline and returns the file.

UC-7 View Email Notifications

Actor: Admin, Seller

Task: Navigate to account email inbox.

Result: AccountService returns stored AccountEmail records for the logged-in user.

UC-8 Talk to connected users (admin/seller) on a chat hub

Actor: Admin, Seller

3 Design Phase

3.1 Architecture Overview

The application follows a microservices architecture. Each service is an independent ASP.NET Core Web API project with its own SQL Server database. The Blazor WebAssembly client communicates with services over HTTPS. Services communicate with each other via plain internal HTTP calls (no JWT required for service-to-service).

Service	Responsibility & Database
AccountService	Authentication (login/register), account management, email notifications storage. DB: AccountDb (Accounts, AccountEmails).
InventoryService	Items, StockUnits, Stocks CRUD, soft-delete/restore logic, data export. DB: InventoryDb.
OrderService	Orders, OrderLines, Receipts, ReceiptLines. DB: OrderDb.
NotificationService	Receives WarehouseEvents from other services, calls AccountService to persist email notifications and handles the chat hub SignalR websocket connection.
Warehouse.Client	Blazor WebAssembly SPA — no server-side rendering. Calls each service via typed HttpClient.
Warehouse.Shared	Shared library: DTOs, Commands/Queries, Result<T>, enums, ValidationBehaviour, JwtExtensions, CurrentUser.

3.2 Authentication JWT Flow

A single shared secret is configured in every service's appsettings.json under CreateJWT:Token. The Warehouse.Shared.Auth library exposes AddSharedJwtAuth() which each service calls once in Program.cs to register JWT Bearer validation. Only AccountService generates tokens; all other services only validate them.

- Client sends login request to AccountService (no token needed).
- AccountService returns a signed JWT string.
- Blazor client stores the token in localStorage via Blazored.LocalStorage.
- JwtAuthProvider reads the token and updates the CascadingAuthenticationState.
- AuthTokenHandler (DelegatingHandler) attaches Authorization: Bearer <token> to every outgoing HttpClient call automatically.
- InventoryService / OrderService validate the token using the shared secret — no call to AccountService needed.

3.3 CQRS with MediatR

All service separates read and write operations into distinct MediatR request types:

Type	Example
Command (write)	CreateItemCommand, UpdateItemCommand, DeleteItemCommand, LoginCommand, RegisterAccountCommand

Query (read)	GetAllItemsQuery, GetItemByIdQuery, GetStockQuery, GetOrdersByCustomerQuery
Pipeline Behaviour	ValidationBehaviour<TRequest, TResponse> runs FluentValidation before every handler; returns Result.MultipleFails on errors.

Controllers are thin they map the incoming DTO to a Command/Query, call mediator.Send(), and unwrap the Result<T>

3.4 Result<T> & Validation

All handlers return Result or Result<T> (defined in Warehouse.Shared.Common):

- Result.Success() / Result<T>.Success(value) operation succeeded.
- Result.Fail(msg) system-level error (single message).
- Result.MultipleFails(failures) FluentValidation field errors as Dictionary<string, List<string>>.

The Blazor client inspects IsSuccessful, then renders FieldErrors inline per field or shows ErrorMessage as a banner alert.

3.5 Design Patterns

Template Method — Export Pipeline

BaseExporter<T> defines the skeleton: Transform() is abstract; Export() seals the flow. Subclasses implement only the format-specific transformation:

- CsvExporter<T> reflects properties via Type.GetProperties(), builds comma-separated output.
- JsonExporter<T> uses System.Text.Json with WriteIndented = true.
- XmlExporter<T> uses XmlSerializer.

The base class owns the orchestration; subclasses cannot reorder the steps.

Command Pattern Blazor UI Actions

Every button click in the Blazor client constructs a command object (shared DTO) and POSTs it to the backend. The controller maps it to a MediatR IRequest<Result<T>> and dispatches via mediator.Send(). This decouples UI event handling from service logic.

Observer Pattern Event Notification

InventoryService and OrderService publish a WarehouseEvent (containing AccountId, AccountEmail, Action, Description) to NotificationService via HTTP POST after state-changing operations. NotificationService forwards the payload to AccountService's internal endpoint, which persists an AccountEmail record. No JWT is required on internal endpoints.

Strategy (preserved from Assignment 2)

IExportStrategy and concrete ExportCSV / ExportJSON / ExportXML classes are retained inside the Template Method hierarchy as the format-specific Transform() implementations.

Repository / Unit of Work (preserved)

Each service's EF DbContext acts as the unit of work. DbSet<T> properties act as typed repositories. Each service owns only the entities relevant to its bounded context.

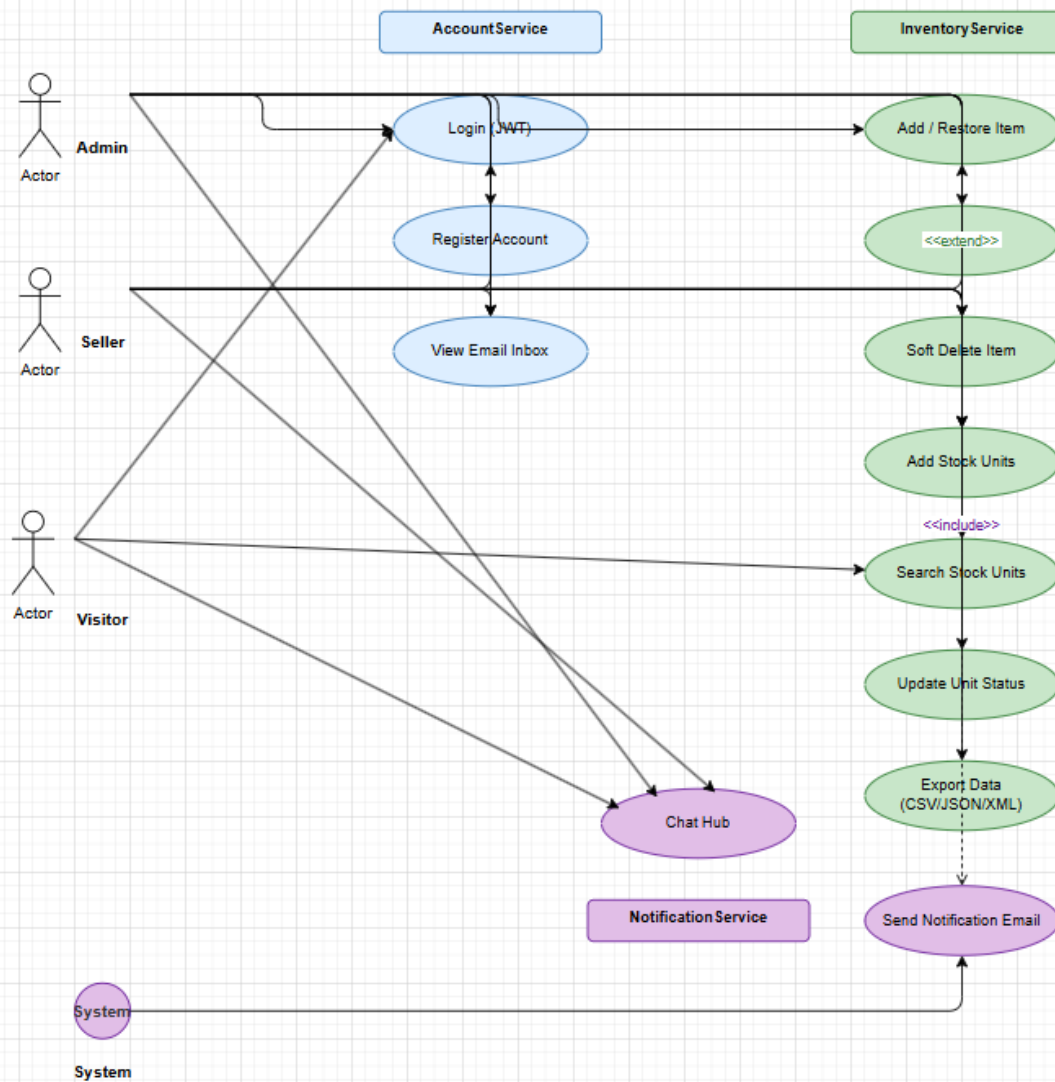
3.6 Entity-Relationship Overview (per service)

Service DB	Tables
AccountDb	Accounts (Id, Email, PasswordHashed, AccountType, IsDeleted, CreatedAt), AccountEmails (Id, AccountId FK, EmailContent)
InventoryDb	Items (Id, Name, Description, ReferencePricePerItem, IsDeleted, CreatedAtTime), StockUnits (Id, ItemId FK, SerialNumber, CurrentPrice, Note, Status, IsDeleted), Stocks (Id, ItemId FK, Quantity >= 0)
OrderDb	Orders (Id, CustomerId, SellerId, Status, CreatedAtTime), OrderLines (Id, OrderId FK, StockUnitId, Quantity), Receipts (Id, OrderId FK, Subtotal, Tax, Total, BuyerEmail), ReceiptLines (Id, ReceiptId FK, UnitPrice, LineTotal, Quantity)

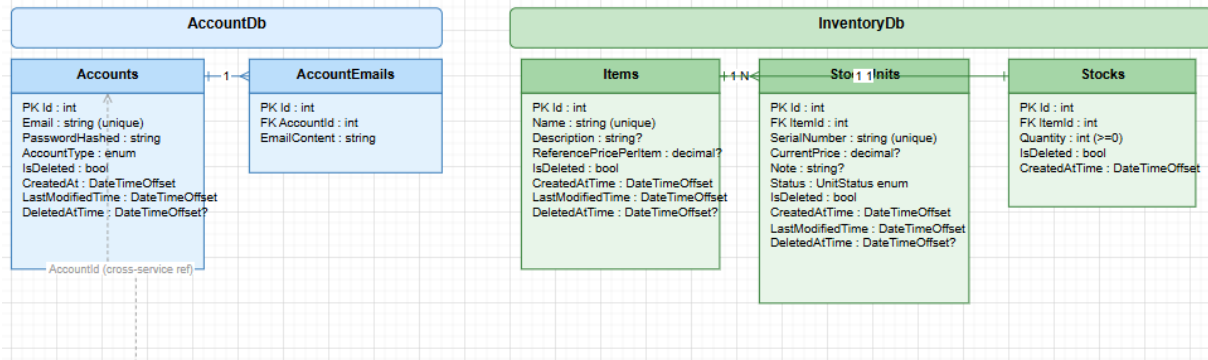
Diagrams (ER, Use Case, Class, Sequence) are included.

Use Case Diagram - Warehouse System

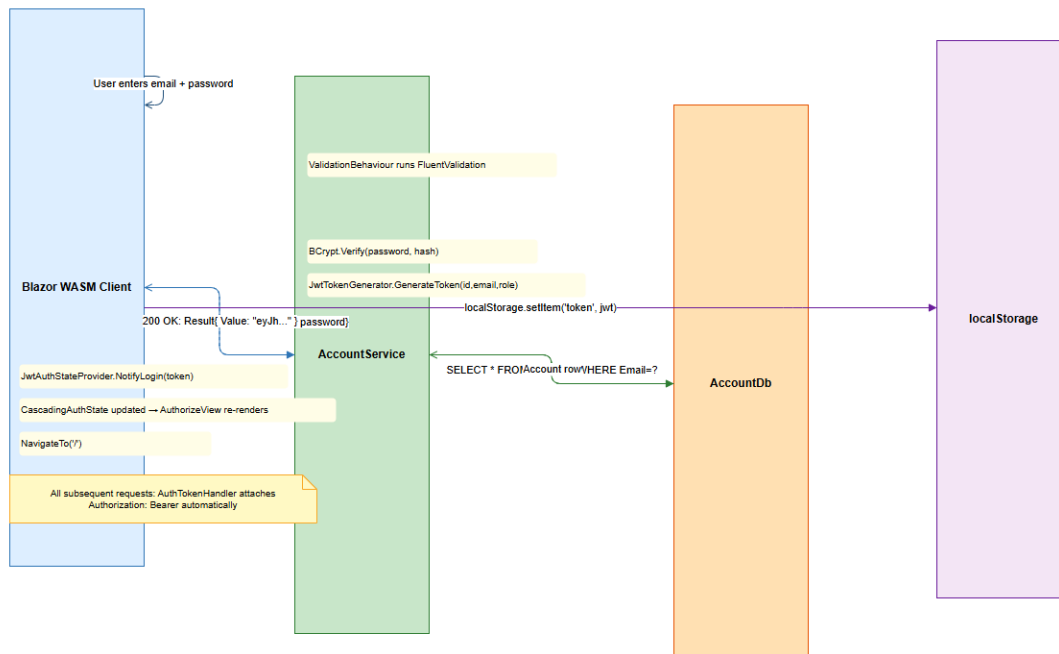
Warehouse System



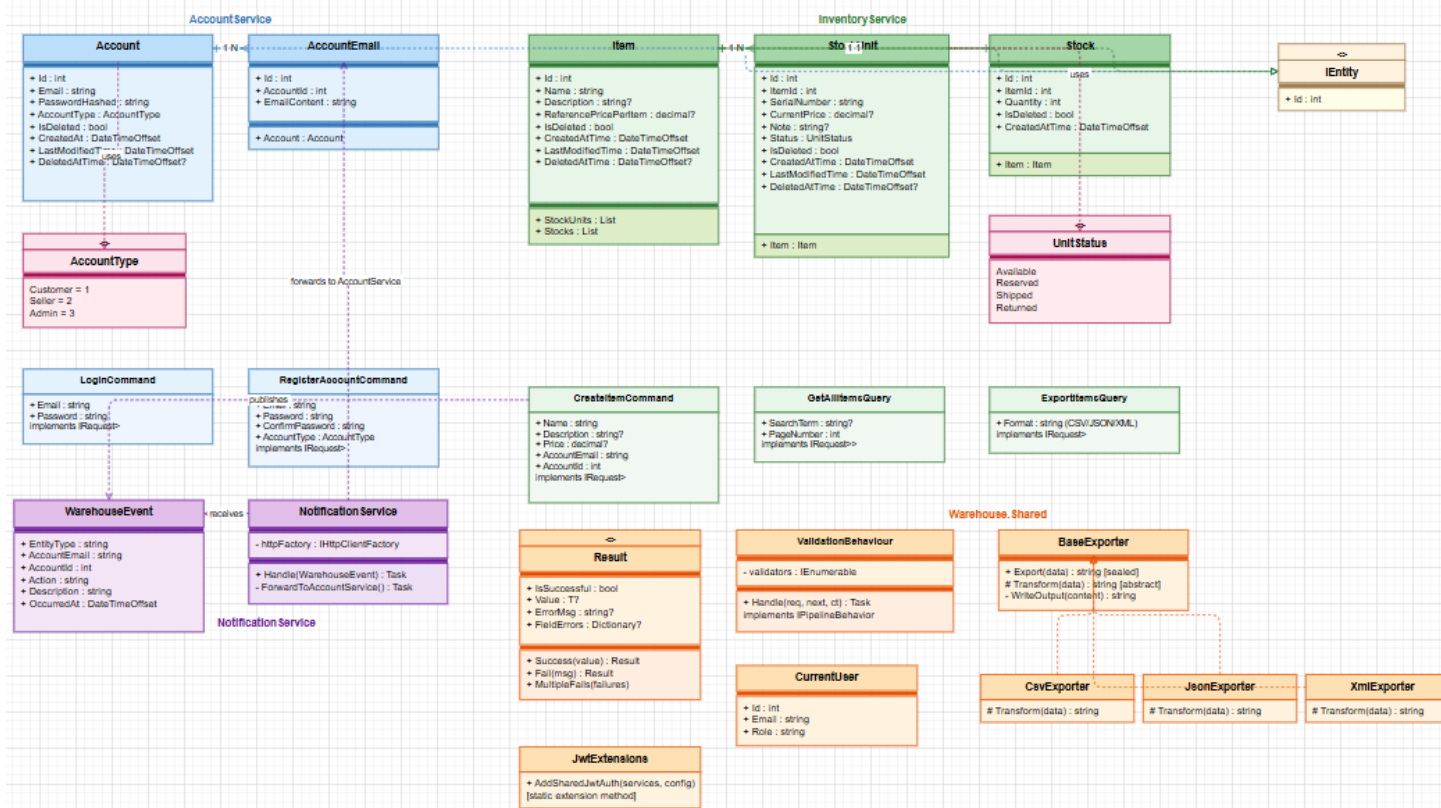
ER Diagram - Warehouse System (Assignment 3 - Per-Service DBs)



Sequence Diagram - Login with JWT



Core Domain Class Diagram — Warehouse System (Assignment 3 — Microservices)



4 Implementation Phase

4.1 Tools and Technologies

Technology	Purpose
C# / .NET 10	Primary language and runtime
ASP.NET Core Web API	Each microservice exposes a REST API
Blazor WebAssembly	SPA frontend runs entirely in the browser
Entity Framework Core	ORM with code-first migrations, per-service DbContext
MediatR SignalR	CQRS dispatches Commands and Queries to handlers Websocket connection server-client
FluentValidation	Declarative validation rules, integrated via ValidationBehaviour
AutoMapper	DTO Entity Command mapping
JWT Bearer (Microsoft.AspNetCore.Authentication.JwtBearer)	Stateless authentication shared across all services
Blazored.LocalStorage	Token persistence in the browser
BCrypt.Net	Password hashing in AccountService
SQL Server	Relational database (one per service)
Bootstrap 5	UI styling for Blazor components

4.2 Solution Structure

- AccountService/ — Application/Commands, Application/Queries, Controllers, Infrastructure/DbRepository, Migrations
- InventoryService/ — Application/Commands, Application/Queries, Application/Export, Controllers, Infrastructure/DbRepository, Migrations
- OrderService/ — Application/Commands, Application/Queries, Controllers, Infrastructure/DbRepository, Migrations
- NotificationService/ — Application/Commands, Controllers
- Warehouse.Client/ — Pages (Login, Register, Items, Stock, Orders, Export), Auth (JwtAuthProvider, AuthTokenHandler), Services (typed API clients)
- Warehouse.Shared/ — Common/Result.cs, Common/ValidationBehaviour.cs, Auth/JwtExtensions.cs, Auth/CurrentUser.cs, DTOs/, Enums/

4.3 JWT Authentication Implementation Detail

Warehouse.Shared.Auth.JwtExtensions.AddSharedJwtAuth(IConfiguration) registers JwtBearer validation using the shared secret. Every service calls this once in Program.cs followed by

app.UseAuthentication() and app.UseAuthorization(). AccountService additionally registers JwtTokenGenerator (scoped) which produces tokens on successful login.

The Blazor client uses AuthTokenHandler (DelegatingHandler) registered on every named HttpClient. After login, JwtAuthProvider.NotifyLogin(token) saves the token to localStorage and signals CascadingAuthenticationState to re-evaluate, updating AuthorizeView components and @attribute [Authorize] pages immediately.

4.4 Soft-Delete & Restore Logic (New in Assignment 3)

When CreateItemCommandHandler receives a new item name, it first queries InventoryDb for an existing record matching that name with IsDeleted = true. If found, the handler sets IsDeleted = false, clears DeletedAtTime, updates LastModifiedTime, and returns the restored item's DTO. If not found, a new Item row is inserted. This prevents duplicate catalogue entries and preserves historical serial numbers linked to the original item.

4.5 Inter-Service Communication

Service-to-service HTTP calls use named HttpClients registered without the AuthTokenHandler (no JWT). Internal endpoints are not decorated with [Authorize]. The notification flow is:

- InventoryService or OrderService handler completes a write operation.
- Handler resolves IHttpConnectionFactory.CreateClient("NotificationService") and POSTs a WarehouseEvent containing AccountId, AccountEmail, Action, and Description.
- NotificationService handler receives the event and POSTs { AccountId, AccountEmail, Content} to AccountService's /api/account/internal/save-email endpoint.
- AccountService persists in an AccountEmail row. No JWT validation occurs on internal endpoints.

4.6 User Manual

Login

Navigate to /login. Enter email and password. On success, the navbar updates show the current user's email and role. Field-level validation errors from the server are displayed in line beneath each input.

Register

Navigate to /register. Admin users see an Account Type selector; non-admins always register as Customer. Password confirmation is validated on server-side by FluentValidation.

Items

The item list displays all active product types. Admin and Seller can create items — if a deleted item with the same name exists, it is restored automatically. Admin can soft-delete items.

Stock Units

Search by product name or partial serial. Add stock by selecting a product and entering a quantity. Change unit status via dropdown.

Export

Select a format (CSV / JSON / XML) from the dropdown and click Export. The file is generated on server-side via the Template Method pipeline and downloaded by the browser.

Email Inbox

Accessible by Admin and Seller. Shows all notification emails stored for the logged-in account.

Chat Page

Logged accounts can talk to each other via chat hub. User sends name and text, and all connected clients on the web see the messages.

5 Testing and Validation

5.2 Manual Integration Tests

Scenario	Expected Result
Login with valid credentials	JWT returned; navbar shows email; protected pages accessible.
Login with wrong password	400 Bad Request; error message displayed on login page.
Register with mismatched passwords	Field error shown under Confirm Password; no account created.
Create item with duplicate deleted name	Item restored; no duplicate row in DB.
Add 3 stock units to an item	3 StockUnit rows created with unique serials; Stock.Quantity += 3.
Export items as CSV	File downloaded; headers and rows match current Items data.
Access /admin page as Seller	403 Forbidden; redirected to Not Authorized page.
Notification after item creation	AccountEmail row created in AccountDb for the logged-in user.